

Counting Minimalist Polyomino Containers: Exhaustive Enumeration of A352029(8)

Peter Exley
pexley-math@github
April 2026

Abstract. We extend OEIS sequence A352029, which counts the number of distinct free polyominoes of minimum size that contain all free n -ominoes as subshapes. Using SAT enumeration with Counter-Example Guided Abstraction Refinement (CEGAR), we prove $a(8) = 7$: there are exactly 7 distinct 20-cell polyominoes that contain all 369 free octominoes. This resolves the first open term of the sequence. The result was independently verified by two verifiers with disjoint code paths and local-optimality checks. We observe a dramatic phase transition in the sequence: $a(7) = 204$ collapses to $a(8) = 7$, driven by exponential growth in the number of pieces overwhelming the sub-quadratic growth of container size.

Keywords: polyomino, minimum common superform, universal container, SAT solver, CEGAR, enumeration

MSC: 05B50, 52C20, 68R05

1. Introduction

1.1 Problem definition

For a positive integer n , let $P(n)$ denote the set of all free n -ominoes (polyominoes with n cells, considered equivalent under translation, rotation, and reflection). Define the *minimum container size* A327094(n) as the fewest cells in a polyomino that contains every member of $P(n)$ as a subshape. A352029(n) then counts the number of distinct free polyominoes with exactly A327094(n) cells that achieve this universal containment property. We call such polyominoes *minimalist containers*.

1.2 Prior work

The sequence A352029 was established by John Mason (2022) with terms $a(1)$ through $a(7)$. The companion sequence A327094 (minimum container sizes) was created by Peter Kagey (2019). The problem of finding minimum common superforms dates to T. R. Dawson (*Fairy Chess Review*, 1942). Alexandre Owen Muniz maintains a webpage on polyomino common superforms with a Python solver using a greedy algorithm, though it does not enumerate all minimalist containers.

1.3 Contributions

Theorem 1. $a(8) = 7$. There are exactly 7 distinct free 20-cell polyominoes containing all 369 free octominoes.

Additionally:

1. **Lower bound:** $a(9) \geq 111$, with 111 distinct 26-cell containers found.

2. **Structural observation:** The sequence exhibits a phase transition driven by constraint density.
3. **Method:** A CEGAR-based SAT enumeration solver with dual-solver selection, independently verified.

2. Definitions and Conventions

A *free polyomino* is a connected set of unit squares on the integer lattice $\mathbb{Z}^2$, considered up to translation, rotation, and reflection (the dihedral group D_4). Two polyominoes are *equivalent* if one can be transformed into the other by an element of D_4 composed with a translation.

A polyomino C *contains* a polyomino P if there exists a rigid motion (element of D_4 plus translation) mapping P into a subset of C .

A *minimalist n -omino container* is a free polyomino C with $|C| = A327094(n)$ cells such that C contains every free n -omino. $A352029(n)$ counts the number of distinct minimalist n -omino containers.

We use (r, c) coordinates with r increasing downward and c increasing rightward. The *bounding box* of a polyomino is the smallest $H \times W$ rectangle enclosing it.

3. Method

3.1 Partition-based decomposition

For fixed n , we know $k = A327094(n)$. We enumerate all valid bounding boxes (H, W) with $H \leq W$ in which a k -cell container might exist. A partition (H, W) is valid if:

- $H \cdot W \geq k$ (enough cells),
- $H + W - 1 \leq k$ (spanning tree bound: a connected polyomino that touches all four borders of an $H \times W$ box needs at least $H + W - 1$ cells; this is enforced by the border-touching constraint below),
- piece-fit check: every free n -omino fits in at least one orientation within $H \times W$,
- triple-piece overhead bound: certain piece combinations force additional empty cells.

For $n = 8$, this yields 36 valid partitions.

3.2 SAT encoding

For each partition (H, W) , we encode the container search as a Boolean satisfiability problem. Cell variables $x_{r,c}$ indicate whether cell (r, c) is occupied. Constraints:

1. **Cardinality:** exactly k cells occupied (totalizer encoding).
2. **Border-touching:** at least one cell in each border row/column.
3. **No-gap:** no empty row or column may separate two occupied rows or columns within the bounding box. Formally: if cell (r, c) is occupied and $r > 0$, then at least one cell in row $r - 1$ must be occupied; similarly for columns. Combined with border-touching, this ensures the occupied cells span the bounding box contiguously.
4. **Local connectivity:** every occupied cell has at least one occupied neighbor.
5. **Piece containment:** for each free n -omino P , at least one valid placement of P must use only occupied cells. This is the key constraint – it requires a disjunction over all placements of P .

6. **Symmetry breaking:** lex-leader constraints for vertical flip, horizontal flip, 180-degree rotation, and (for square grids) 90-degree rotation. These eliminate symmetric duplicates from the search without losing canonical solutions.

3.3 CEGAR

With 369 free octominoes and thousands of placements each, encoding all piece constraints upfront creates a formula with millions of clauses. Counter-Example Guided Abstraction Refinement (CEGAR) addresses this by starting with only the 20 hardest-to-place pieces (those with fewest valid placements). The solver finds candidate containers satisfying these partial constraints, then verifies each candidate against all 369 pieces outside the SAT solver. Failed pieces are added incrementally, preserving learned clauses. This reduces formula size by 10–100 $\times$ and gives 10–50 $\times$ speedup on satisfiable partitions.

3.4 Enumeration

Solutions are enumerated by iterating: find a solution, add a blocking clause (at least one occupied cell must change), solve again. Each solution is canonicalized under D_4 equivalence. Solutions are deduplicated across partitions.

3.5 Connectivity enforcement

Global connectivity is enforced lazily. After finding a candidate with exactly k cells, we check connectivity via BFS. If the candidate is disconnected, we add a clause removing the smallest component, forcing the solver to place those cells elsewhere. This converges in practice because the local connectivity and no-gap constraints heavily constrain the search.

3.6 Dual-solver selection

Based on partition characteristics, we select between Glucose 4.2 (faster for satisfiable instances, measured 46–71 $\times$ speedup over CaDiCaL on A352029 SAT partitions) and CaDiCaL 1.9.5 with aggressive restarts (`restartint=5`; better for unsatisfiable near-square partitions with aspect ratio > 0.6 and slack ≤ 2).

4. Results

4.1 Proved values

n	$a(n)$	k	$ P(n) $	Time (s)	Parts.	Source
1	1	1	1	–	–	Mason ‘22 (conf.)
2	1	2	1	–	–	Mason ‘22 (conf.)
3	2	4	2	0.2	1	Mason ‘22 (conf.)
4	2	6	5	0.1	1	Mason ‘22 (conf.)
5	2	9	12	0.2	3	Mason ‘22 (conf.)
6	14	12	35	0.4	8	Mason ‘22 (conf.)
7	204	17	108	45.6	24	Mason ‘22 (conf.)
8	7	20	369	45.7	36	this work

Table 1: Proved values of A352029(n). $k = A327094(n)$ is the container size; $|P(n)| = A000105(n)$ is the number of free n -ominoes. Timings on AMD Ryzen 5 5600, 8 workers.

The similar runtimes for $n = 7$ (45.6 s) and $n = 8$ (45.7 s) reflect a tradeoff: $n = 8$ has more partitions (36 vs 24) but each partition resolves faster because higher constraint density causes CEGAR to prove UNSAT with fewer pieces (typically 20 of 369).

All values for $n = 3$ through 8 were independently verified by two verification scripts using disjoint code paths (matrix-based and complex-number coordinate transformations). Both verifiers include local-optimality checks confirming that every cell in every container is essential.

4.2 The 7 minimalist containers for $n = 8$

All 7 containers have bounding box 4×8 (6 solutions) or 5×8 (1 solution). The containers all share a common structural pattern: a full top row of 8 cells, a nearly-full second row, and two 3-cell blocks in the lower rows.

4.3 Lower bound for $a(9)$

$a(9) \geq 111$. With $k = A327094(9) = 26$ cells and $|P(9)| = A000105(9) = 1285$ free 9-ominoes, the 111 known containers were found across four bounding box partitions: 5×9 (10 solutions), 6×9 (97 solutions), 6×10 (1 solution), and 7×9 (3 solutions). The remaining partitions (all expected UNSAT) require extended computation.

5. Structural Observations

5.1 Phase transition

The most striking feature of A352029 is its non-monotonicity. The sequence rises from $a(5) = 2$ through $a(6) = 14$ and $a(7) = 204$, then crashes to $a(8) = 7$. This pattern is characteristic of constraint satisfaction problems near the satisfiability threshold.

The *constraint density* $d(n) = |P(n)| / A327094(n)$ measures how many pieces must fit per cell:

n	$d(n)$	$a(n)$	Regime
1–5	< 2	1–2	Under-constrained
6	2.9	14	Moderate
7	6.4	204	Peak
8	18.5	7	Over-constrained
9	49.4	≥ 111	Extreme

Table 2: Constraint density $d(n) = |P(n)| / A327094(n)$ and solution count.

The peak at $n = 7$ occurs because constraint density is high enough to be selective but not so high as to be prohibitive. The partial data for $a(9) \geq 111$ shows that even at extreme density, solutions persist in the narrowest bounding boxes (5×9 and 6×9 account for 107 of 111 known solutions). The behavior is non-monotonic rather than a simple decline – the count depends on the interplay between piece diversity and geometric flexibility in specific bounding boxes.

5.2 Cross-lattice universality

The same non-monotonic pattern appears in the triangular grid analog (polyiamond minimalist containers): $a(1..10) = 1, 1, 1, 1, 1, 4, 28, 388, 672, 563$, with the peak at $n = 9$. The peak location correlates with neighbor count: the square grid (4 neighbors) peaks at $n = 7$, the triangular grid (3 neighbors) peaks at $n = 9$. Higher neighbor count means more orientations per piece, faster density growth, and an earlier phase transition.

5.3 Bounding box distribution

For $n = 8$, all solutions concentrate in narrow bounding boxes (4×8 and 5×8). The remaining 34 partitions (6×8 through 9×12) are all UNSAT. This concentration in narrow boxes is consistent with the observation that minimalist containers must pack cells efficiently, favoring elongated shapes that accommodate the I-piece (1×8) along one dimension.

6. Open Problems

1. Determine $a(9)$ exactly. Our lower bound is $a(9) \geq 111$. Proving this requires exhaustive UNSAT proofs for approximately 50 bounding box partitions, several of which exceed current computational limits.
2. Can the bounding box dimensions of minimalist containers be bounded as a function of n ? For small n , all minimalist containers are narrow ($H \leq 5$ for $n \leq 8$), but it is unclear whether this continues.
3. The cross-lattice phase transition observation suggests a universal phenomenon. Is there a theoretical framework predicting the peak location from the growth rate of the polyform count and the neighbor count of the lattice?

7. Reproducibility

The solver, verification scripts, and all data are available at:

<https://github.com/pexley-math/oeis-A352029>

To reproduce: `python code/solve_a352029.py --n 8 --workers 8`

To verify: `python code/verify_v1.py 8` and `python code/verify_v2.py 8`

Dependencies: Python 3.10+, PySAT (python-sat). Optional: polyform_enum (Cython, for faster enumeration).

Hardware: AMD Ryzen 5 5600 (6-core / 12-thread), 16 GB RAM. $a(8)$ completes in under 1 minute.

References

1. J. Mason, OEIS A352029 (2022). <https://oeis.org/A352029>
2. P. Kagey, OEIS A327094 (2019). <https://oeis.org/A327094>
3. T. R. Dawson, *Fairy Chess Review*, Vol. 5, No. 4 (1942).
4. A. O. Muniz, “Polyomino Common Superforms,” Puzzle Zapper. <https://puzzlezapper.com/aom/mathrec/polycover.html>